Breast Cancer Diagnosis - Five-Model Study

Breast Cancer Diagnosis - Five-Model Classification Study

Machine Learning Assignment 2 · Classification models, evaluation metrics and a Streamlit deployment.

---

a. Problem statement

Early and accurate discrimination between **malignant** and **benign** breast tumours is a high-stakes classification task. Given 30 numeric measurements computed from a digitised image of a fine-needle aspirate (FNA) of a breast mass, the goal is to predict whether the tumour is malignant (`1`) or benign (`0`).

This project implements and compares **five classifiers** on the *same* dataset - Logistic Regression, Decision Tree, K-Nearest Neighbours, Gaussian Naive Bayes and Support Vector Machine - using six evaluation metrics (Accuracy, AUC, Precision, Recall, F1, MCC), and deploys the trained models in a Streamlit web app that evaluates any uploaded test CSV **without re-training**.

Because a missed cancer (false negative) is costlier than a false alarm, Recall for the malignant class is read alongside the overall metrics when choosing a winner.

b. Dataset description [1 mark]

Item	Value
Name	Breast Cancer Wisconsin (Diagnostic)
Source	UCI Machine Learning Repository (distributed with scikit-learn as `load_breast_cancer`)
Problem type	Binary classification
Instances	**569** (requirement: ≥ 500)
Features	**30** numeric (requirement: ≥ 12)
Target	`diagnosis`: **1 = malignant (212)**, **0 = benign (357)** - 37.3 % positive class
Missing values	None

The 30 features are the **mean**, **standard error** and **"worst" (mean of the three largest values)** of 10 cell-nucleus characteristics: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension.

Pre-processing / split

- scikit-learn encodes malignant = 0; the label was flipped so that malignant is the positive class (1).
- Stratified 80 / 20 train-test split (`random_state = 2026`): 455 training rows, **114 test rows** (72 benign / 42 malignant). The test split is saved as `test_data.csv`.
- `StandardScaler` is part of every model's `Pipeline` (fit on training data only), so the saved `.joblib` files already contain the preprocessing.

Model settings (see `train_models.py`)

Model	Key hyper-parameters
Logistic Regression	`C=1.0`, `max_iter=2000`
Decision Tree	`max_depth=5`, `min_samples_leaf=3`
KNN	`n_neighbors=7`, distance weighting
Naive Bayes	`GaussianNB` (defaults)
Support Vector Machine	RBF kernel, `C=2.0`, `gamma="scale"` (AUC from `decision_function`)

c. GitHub repository link [1 mark]

GitHub: `https://github.com/<your-username>/<your-repo-name>` ← *replace with your repo URL*

Live Streamlit app: `https://<your-app-name>.streamlit.app` ← *replace after deployment*

```
bc-diagnosis-classifier/
|-- app.py                 Streamlit application
|-- train_models.py        Trains the 5 models, writes metrics, saves artifacts
|-- requirements.txt
|-- README.md
|-- test_data.csv          Hold-out test set (114 rows: 30 features + diagnosis)
|-- full_dataset.csv       Complete dataset (reference)
|-- model/
|      ├── logistic_regression.joblib
|      ├── decision_tree.joblib
|      ├── knn.joblib
|      ├── naive_bayes.joblib
|      ├── support_vector_machine.joblib
|      ├── feature_names.json
|      └── metrics.csv
```

Run locally

```
pip install -r requirements.txt
python train_models.py        # optional: regenerates models, metrics and test_data.csv
streamlit run app.py
```

d. Models used [5 marks]

Metrics on the 114-row hold-out test set (positive class = malignant):

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9737	0.9987	0.9756	0.9524	0.9639	0.9433
Decision Tree	0.9737	0.9785	0.9756	0.9524	0.9639	0.9433
KNN	0.9737	0.9970	0.9535	0.9762	0.9647	0.9439
Naive Bayes	0.9561	0.9964	0.9302	0.9524	0.9412	0.9064
Support Vector Machine	**0.9825**	**1.0000**	0.9545	**1.0000**	**0.9767**	**0.9633**

Confusion-matrix counts behind these numbers:

Model	TN	FP	FN	TP
Logistic Regression	71	1	2	40
Decision Tree	71	1	2	40
KNN	70	2	1	41
Naive Bayes	69	3	2	40
Support Vector Machine	70	2	0	42

Observations on model performance [3 marks]

ML Model Name	Observation about model performance
Logistic Regression	Very strong, simple baseline: 97.4 % accuracy and the second-best AUC (0.9987). The classes are almost linearly separable after scaling. Highest precision (tied), but it missed 2 malignant cases (Recall 0.952).
Decision Tree	Same confusion matrix as Logistic Regression, so identical accuracy/F1/MCC, but clearly the lowest AUC (0.9785): a depth-5 tree outputs coarse leaf probabilities, so it ranks cases less finely. Also the least stable model (high variance), and its result could change with a different split.
KNN	97.4 % accuracy with better Recall (0.976) than LR/DT - only 1 missed malignant case - at the cost of 2 false alarms. Depends on feature scaling (done in the pipeline); prediction is slower since it stores the training set.
Naive Bayes	Weakest on accuracy (95.6 %), precision (0.930) and MCC (0.906), with the most false positives (3). The independence assumption is violated because many features (radius, perimeter, area) are highly correlated. Still competitive on AUC (0.9964) and the cheapest to train.
Support Vector Machine	Best on Accuracy, AUC, Recall, F1 and MCC. It caught **all 42 malignant cases** (0 false negatives) with only 2 false positives; the RBF kernel captures the mildly non-linear boundary. Its precision (0.9545) is slightly below LR/DT.
Overall, Winner for your dataset?	**Support Vector Machine (RBF)** - highest Accuracy (0.9825), AUC (1.0000), Recall (1.000), F1 (0.9767) and MCC (0.9633), and zero missed cancers, which matters most clinically. *Caveat:* the test set has only 114 rows, so one case moves accuracy by ~0.9 pp; differences between SVM, KNN and Logistic Regression are small and should be confirmed with cross-validation before drawing strong conclusions.

Streamlit app features

Required feature	Where in the app
CSV dataset upload (test data)	Sidebar → *Upload test CSV* (validates columns, handles bad rows)
Model selection dropdown	Sidebar → *Choose a classifier* (5 models)
Evaluation metrics	Six metric cards (Accuracy, AUC, Precision, Recall, F1, MCC)
Confusion matrix / classification report	Both shown, plus ROC curve
Results of all models	Tab *Compare all 5 models* (table + bar chart + CSV download)

The uploaded CSV must contain the 30 feature columns of `full_dataset.csv` and the `diagnosis` column (`1` = malignant, `0` = benign; `M`/`B` also accepted). The app only calls `predict` / `predict_proba` / `decision_function` on the stored models - nothing is re-fitted.

Notes on reproducibility

- Package versions in `requirements.txt` match the environment used to train the saved models. If your Streamlit Cloud or lab environment uses a different scikit-learn version, re-run `python train_models.py` there (or change the pin) and commit the regenerated `model/` folder to avoid unpickling errors.
- Everything is seeded (`SEED = 2026`), so re-running the script reproduces the table above.